

Question 1:

A **node** is one individual element in a tree that stores a value and may point to other nodes.

A **root** is the topmost node of the tree; it has no parent.

A **leaf** is a node with no children.

Height of a tree is the length of the longest path from the root to a leaf. In many classes, this is counted in **edges**.

For the given tree:

- Root = **8**
- Left child of 8 = **3**
- Right child of 8 = **10**
- Children of 3 = **1** and **6**

So:

- **Root:** 8
- **Leaves:** 1, 6, 10
- **Height:** 2 edges

Question 2:

For the same tree:

Inorder traversal:

Left, Root, Right

1, 3, 6, 8, 10

Preorder traversal:

Root, Left, Right

8, 3, 1, 6, 10

Postorder traversal:

Left, Right, Root

1, 6, 3, 10, 8

Why inorder of a BST is sorted

In a Binary Search Tree, all values in the left subtree are smaller than the node, and all values in the right subtree are larger. Inorder traversal visits the left subtree first, then the node, then the right subtree, so the values come out in increasing order.

Question 3:

BST Rule

In a Binary Search Tree, every node must follow this rule:

- values in the **left subtree** are smaller than the node's value
- values in the **right subtree** are greater than the node's value

This rule must be true for every node in the tree.

Python function: insert into BST

TreeNode class

class TreeNode:

def __init__(self, data):

self.data = data

self.left = None

self.right = None

Function to insert a value into the BST

def insert(node, data):

"""

Insert 'data' into a Binary Search Tree

and return the root node.

"""

If the tree is empty, create a new node

if node is None:

return TreeNode(data)

Go to the left subtree if data is smaller

if data < node.data:

node.left = insert(node.left, data)

Otherwise, go to the right subtree

else:

node.right = insert(node.right, data)

return node

Inorder traversal (for verification)

def inorder(node):

if node:

inorder(node.left)

print(node.data, end=" ")

inorder(node.right)

----- TEST CASE -----

Create an empty BST

root = None

Insert values in a non-sorted order

values = [8, 3, 10, 1, 6, 14]

for value in values:

root = insert(root, value)

Print values using inorder traversal

print("Inorder Traversal:")

inorder(root)

Output:

1, 3, 6, 8, 10, 14

Question 4:

When deleting a node from a BST, there are **3 cases**:

1. Deleting a leaf node

A leaf has no children, so it can simply be removed.

2. Deleting a node with one child

The node is replaced by its only child, so the tree stays connected.

3. Deleting a node with two children

This is the trickiest case. We cannot just remove the node directly, because that would break the BST structure. So we replace its value with the **inorder successor**.

What is an inorder successor?

The inorder successor is the **smallest value in the node's right subtree**.

Why use it?

We use it because it is the next bigger value in sorted order, so replacing the node with it keeps the BST rule valid.

Question 5:

Time complexity table:

Operation	Average Case (Balanced)	Worst Case (Unbalanced)
Search	$O(\log n)$	$O(n)$
Insert	$O(\log n)$	$O(n)$
Delete	$O(\log n)$	$O(n)$

Brief explanation of one answer

For **search** in a balanced BST, each comparison cuts the remaining tree roughly in half, so the time is **$O(\log n)$** . In an unbalanced tree, the nodes can form a chain, so you may need to check almost every node, which becomes **$O(n)$** .

If values are inserted in already-sorted order

The BST becomes a **skewed tree** or **linked-list-shaped tree**. This happens because every new value keeps going to one side only, so the tree loses balance. Performance gets worse because search, insert, and delete can all take **$O(n)$** instead of **$O(\log n)$** .